МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
СУМСЬКИЙ ДЕРЖАВНИЙ УНІВЕРСИТЕТ
Факультет електроніки та інформаційних технологій
Кафедра комп’ютерних наук

КУРСОВА РОБОТА

з дисципліни «Об’єктно-орієнтоване програмування на мові Java»

на тему:

«Інформаційна система обліку підрозділів і працівників підприємства»

Варіант 4

Виконав:
студент групи _____
Река Є. Д.

Перевірив:
П’ятаченко В. Ю.

Суми – 2026

ЗМІСТ

ВСТУП 3
1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАДАЧІ 5
1.1 Опис предметної області 5
1.2 Постановка задачі 5
1.3 Вибір засобів розробки 6
2 ПРОЄКТУВАННЯ ПРОГРАМНОГО ЗАСОБУ 8
2.1 Структура бази даних 8
2.2 ER-діаграма 9
2.3 Архітектура застосунку 9
2.4 Діаграма класів 11
3 ПРОГРАМНА РЕАЛІЗАЦІЯ 12
3.1 Підключення до бази даних і транзакції 12
3.2 Доступ до даних через JDBC 13
3.3 Бізнес-логіка та перевірка даних 14
3.4 Експорт та імпорт JSON 16
3.5 Графічний інтерфейс користувача 17
3.6 Обробка винятків 20
4 ТЕСТУВАННЯ 21
4.1 Функціональне тестування інтерфейсу 21
4.2 Автоматизовані тести 22
ВИСНОВКИ 24
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ 25
ДОДАТОК А ІНСТРУКЦІЯ ІЗ ЗАПУСКУ 26

ВСТУП

Облік персоналу є однією з базових задач будь-якого підприємства. Керівникам і працівникам відділу кадрів постійно потрібна актуальна інформація про структуру організації: які підрозділи існують, скільки в них працює людей, на яких посадах і з якою заробітною платою. Ведення таких відомостей у паперовому вигляді або в електронних таблицях швидко призводить до дублювання, помилок і суперечностей між документами, а також ускладнює отримання зведених показників.

Метою курсової роботи є розробка настільного застосунку мовою Java для ведення обліку підрозділів підприємства та їхніх працівників зі зберіганням даних у реляційній базі даних PostgreSQL.

Для досягнення мети необхідно розв’язати такі задачі:

– проаналізувати предметну область і визначити перелік сутностей та зв’язків між ними;

– спроєктувати структуру бази даних з обмеженнями цілісності;

– спроєктувати архітектуру застосунку з розподілом відповідальності між класами відповідно до принципів об’єктно-орієнтованого програмування;

– реалізувати доступ до бази даних засобами JDBC з використанням параметризованих запитів PreparedStatement;

– реалізувати графічний інтерфейс користувача на JavaFX з перевіркою введених даних і обробкою винятків;

– реалізувати функції варіанта 4: статистику підрозділів, переведення працівників, розрахунок фонду заробітної плати, експорт та імпорт даних у форматі JSON;

– виконати тестування програми та оформити результати.

Об’єктом розроблення є процес обліку кадрового складу підприємства. Предметом розроблення є програмний засіб, що автоматизує цей облік.

Результатом роботи є застосунок «Облік підрозділів і працівників», вихідний код якого разом зі скриптами бази даних і автоматизованими тестами розміщено в репозиторії GitHub: https://github.com/CCh1e/staff-coursework.

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАДАЧІ

1.1 Опис предметної області

Підприємство складається з підрозділів (відділів). Кожен підрозділ має унікальну назву, розташування (корпус, кабінет) і контактний телефон. У підрозділі працюють працівники, причому кожен працівник у певний момент часу належить рівно до одного підрозділу.

Про працівника зберігаються прізвище, ім’я, по батькові, посада, розмір заробітної плати, дата прийняття на роботу та службова електронна пошта. Електронна пошта є необов’язковою, але якщо вона вказана, то не може повторюватися в різних працівників, тому її можна використовувати як природний ідентифікатор працівника під час обміну даними з іншими системами.

Типові операції користувача системи:

– перегляд переліку підрозділів і чисельності кожного з них;

– перегляд працівників вибраного підрозділу;

– додавання, редагування та видалення підрозділів і працівників;

– масове переведення всіх працівників одного підрозділу до іншого (наприклад, під час реорганізації чи об’єднання відділів);

– аналіз фонду заробітної плати: загальна сума та частка кожного підрозділу;

– резервне копіювання та перенесення даних між базами за допомогою файлу JSON.

Важливим бізнес-правилом є неможливість видалити підрозділ, у якому ще є працівники: спочатку працівників потрібно перевести до іншого підрозділу або звільнити. Це правило забезпечується як у застосунку, так і на рівні бази даних через зовнішній ключ з дією ON DELETE RESTRICT.

1.2 Постановка задачі

Загальні вимоги до програми:

1) графічний інтерфейс користувача на JavaFX;

2) зберігання даних у СКБД PostgreSQL у двох або більше пов’язаних таблицях;

3) вхід у програму з використанням імені користувача та пароля бази даних;

4) перегляд, додавання, редагування та видалення записів;

5) перевірка коректності введених даних і обробка винятків із виведенням зрозумілих повідомлень;

6) робота з базою даних через JDBC з використанням PreparedStatement;

7) розподіл відповідальності між класами відповідно до принципів ООП; користувач не виконує SQL-запити безпосередньо.

Додаткові вимоги варіанта 4:

– **на 75 балів** – вивести перелік підрозділів із кількістю працівників у кожному, при виборі підрозділу показувати список його працівників, забезпечити сортування підрозділів за кількістю працівників;

– **на 85 балів** – реалізувати переведення всіх працівників вибраного підрозділу до іншого підрозділу з оновленням даних у базі; обчислити та відобразити загальний фонд заробітної плати всіх підрозділів і частку кожного підрозділу;

– **на 100 балів** – реалізувати експорт працівників і підрозділів у файл JSON та імпорт даних із файлу JSON.

1.3 Вибір засобів розробки

Для реалізації обрано такі технології:

– мова Java (рівень мови 25) – сучасні можливості мови: записи (record), текстові блоки, вирази switch, var;

– JavaFX 25 – бібліотека для побудови графічного інтерфейсу з таблицями, діалогами та діаграмами;

– PostgreSQL – реляційна СКБД з підтримкою транзакцій, зовнішніх ключів і обмежень CHECK;

– драйвер PostgreSQL JDBC 42.7 – стандартний інтерфейс доступу до бази даних з Java;

– Jackson 2 – серіалізація об’єктів у JSON і назад, модуль jsr310 для дат LocalDate;

– Apache Maven – збирання проєкту та керування залежностями;

– JUnit 6 та Embedded PostgreSQL – автоматизовані тести на справжньому сервері PostgreSQL, що запускається безпосередньо з тестів.

2 ПРОЄКТУВАННЯ ПРОГРАМНОГО ЗАСОБУ

2.1 Структура бази даних

База даних staff_db містить дві таблиці: departments (підрозділи) та employees (працівники). Таблиці пов’язані відношенням «один до багатьох»: один підрозділ може містити багато працівників, кожен працівник належить до одного підрозділу. Структуру таблиць наведено в таблицях 2.1 і 2.2.

Таблиця 2.1 – Структура таблиці departments

Поле	Тип	Обмеження	Опис
id	SERIAL	PRIMARY KEY	Ідентифікатор підрозділу
name	VARCHAR(100)	NOT NULL, UNIQUE	Назва підрозділу
location	VARCHAR(100)	—	Розташування
phone	VARCHAR(20)	—	Контактний телефон

Таблиця 2.2 – Структура таблиці employees

Поле	Тип	Обмеження	Опис
id	SERIAL	PRIMARY KEY	Ідентифікатор працівника
last_name	VARCHAR(50)	NOT NULL	Прізвище
first_name	VARCHAR(50)	NOT NULL	Ім’я
middle_name	VARCHAR(50)	—	По батькові
position	VARCHAR(100)	NOT NULL	Посада
salary	NUMERIC(12,2)	NOT NULL, CHECK (salary >= 0)	Заробітна плата, грн
hire_date	DATE	NOT NULL	Дата прийняття на роботу
email	VARCHAR(100)	UNIQUE	Службова електронна пошта
department_id	INTEGER	NOT NULL, FOREIGN KEY → departments(id) ON DELETE RESTRICT	Підрозділ

Для прискорення вибірки працівників підрозділу та підрахунку чисельності створено індекс idx_employees_department на полі department_id. Для зберігання грошових сум використано тип NUMERIC, який, на відміну від чисел з плаваючою комою, не вносить похибок округлення; у Java йому відповідає клас BigDecimal.

Скрипт створення таблиць міститься у файлі sql/schema.sql, тестові дані (6 підрозділів і 18 працівників) – у файлі sql/data.sql.

2.2 ER-діаграма

ER-діаграму бази даних наведено на рисунку 2.1. Позначення «||» біля таблиці departments і «o{» біля таблиці employees означають, що кожен працівник пов’язаний рівно з одним підрозділом, а підрозділ може мати нуль або більше працівників.

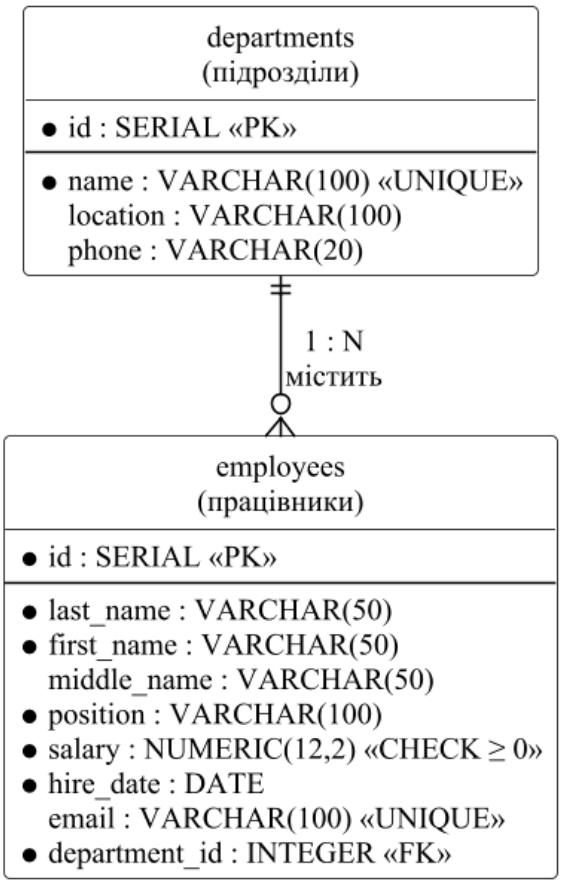


Рисунок 2.1 – ER-діаграма бази даних

2.3 Архітектура застосунку

Застосунок побудовано за багаторівневою архітектурою. Кожен рівень відповідає за свою частину роботи та звертається лише до нижчого рівня, що спрощує супровід і тестування. Пакети проєкту та їхнє призначення наведено в таблиці 2.3.

Таблиця 2.3 – Пакети застосунку

Пакет	Призначення	Основні класи
`model`	Класи предметної області та результати звітів	Department, Employee, DepartmentStats, SalaryFundReport, SortOrder, ImportResult
`exception`	Власні винятки застосунку з повідомленнями українською мовою	AppException, ValidationException, DataAccessException
`db`	Параметри підключення, відкриття з'єднань, транзакції, переклад помилок SQL	ConnectionSettings, Database, SqlErrors
`dao`	Інтерфейси та JDBC-реалізації доступу до таблиць	DepartmentDao, EmployeeDao, JdbcDepartmentDao, JdbcEmployeeDao
`service`	Бізнес-логіка, перевірка даних, обмін JSON	DepartmentService, EmployeeService, JsonExchangeService, Validator
`json`	Формат файлу обміну	ExchangeData
`ui`	Графічний інтерфейс JavaFX	StaffApp, LoginView, MainView, DepartmentsTab, EmployeesTab, SalaryFundTab, діалоги

Інтерфейс користувача не містить жодного SQL-запиту: він викликає лише методи сервісів. Сервіси виконують перевірку даних і бізнес-правил, а потім звертаються до DAO в межах одного з'єднання або однієї транзакції. Методи DAO отримують з'єднання Connection як параметр, завдяки чому кілька операцій різних DAO можна об'єднати в одну транзакцію (це використано під час переведення працівників та імпорту JSON).

Принципи ООП застосовано так:

– **інкапсуляція** – поля моделей закриті й доступні через методи; деталі роботи з JDBC приховані всередині DAO;

– **абстракція і поліморфізм** – сервіси залежать від інтерфейсів DepartmentDao та EmployeeDao, а не від конкретних JDBC-класів;

– **наслідування** – ієрархія винятків від AppException; вкладки інтерфейсу успадковуються від Tab, діалоги – від Dialog;

– **композиція** – клас AppContext створює та зв'язує DAO і сервіси, головне вікно MainView складається з окремих вкладок.

2.4 Діаграма класів

UML-діаграму класів наведено на рисунку 2.2. На ній показано основні класи кожного пакета, їхні відкриті методи та зв’язки: реалізацію інтерфейсів DAO, композицію сервісів у AppContext, залежності інтерфейсу від сервісів і сервісів від DAO.

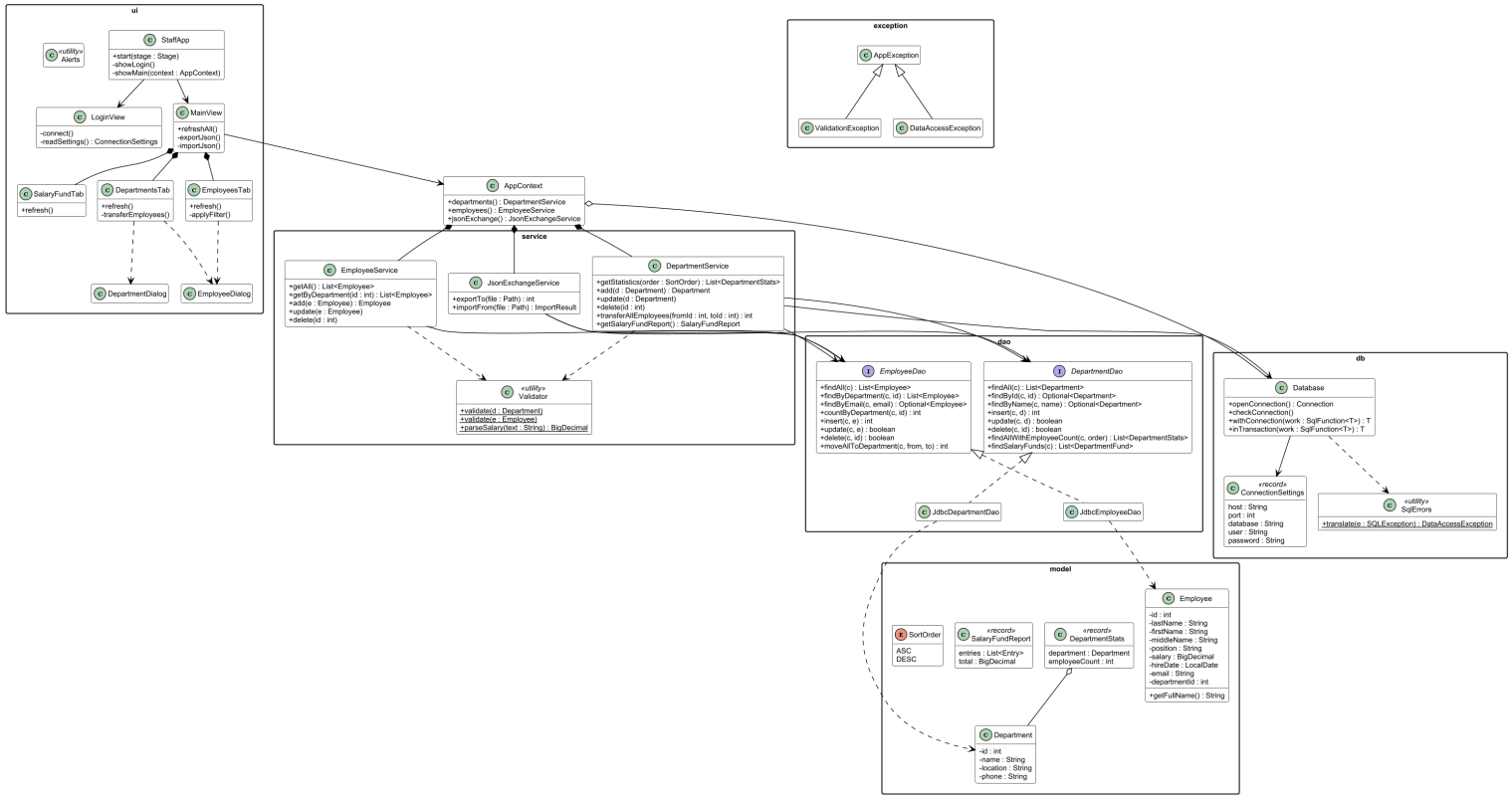


Рисунок 2.2 – UML-діаграма класів застосунку

3 ПРОГРАМНА РЕАЛІЗАЦІЯ

3.1 Підключення до бази даних і транзакції

Під час запуску користувач вводить адресу сервера, порт, назву бази даних, ім’я користувача та пароль PostgreSQL (рисунок 3.1). Перевірка підключення виконується у фоновому потоці (javafx.concurrent.Task), щоб інтерфейс не «зависав». Клас Database відкриває з’єднання через DriverManager і переконується, що в базі є обидві таблиці. Якщо логін або пароль неправильні, сервер недоступний чи бази даних не існує, користувач бачить відповідне повідомлення українською мовою.

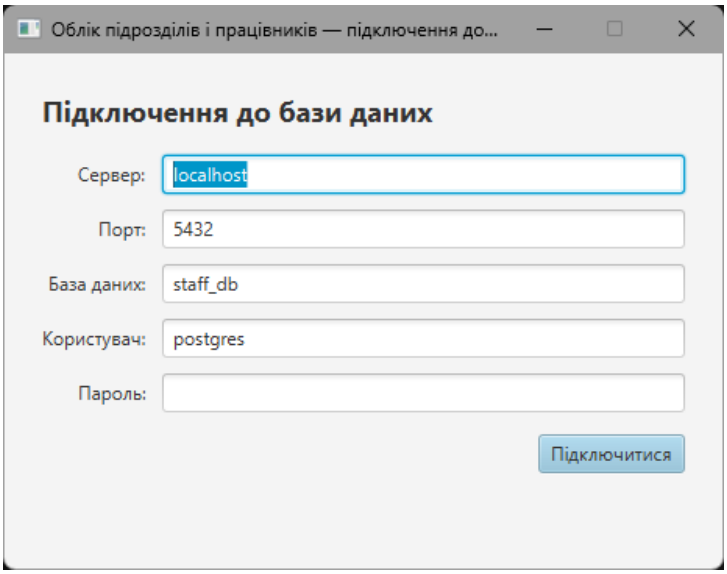


Рисунок 3.1 – Вікно входу

Для операцій, що змінюють кілька записів, використано метод inTransaction (лістинг 3.1). Він вимикає автофіксацію, виконує передану дію та фіксує транзакцію, а за будь-якої помилки відкочує всі зміни.

Лістинг 3.1 – Виконання дії в одній транзакції (клас Database)

```
public <T> T inTransaction(SqlFunction<T> work) {
    try (Connection connection = openConnection()) {
        connection.setAutoCommit(false);
        try {
            T result = work.apply(connection);
            connection.commit();
            return result;
        } catch (SQLException | RuntimeException e) {
            connection.rollback();
            throw e;
        }
    } catch (SQLException e) {
        throw SqlErrors.translate(e);
    }
}
```

3.2 Доступ до даних через JDBC

Усі запити до бази даних виконуються через PreparedStatement із параметрами «?», що унеможливлює SQL-ін’єкції, оскільки введені користувачем значення ніколи не підставляються в текст запиту. Ресурси (PreparedStatement, ResultSet, Connection) закриваються автоматично конструкцією try-with-resources.

Статистику підрозділів для вимоги на 75 балів отримано одним запитом із LEFT JOIN та COUNT (лістинг 3.2). LEFT JOIN гарантує, що підрозділи без працівників теж потраплять до списку з кількістю 0. Напрям сортування ASC або DESC підставляється лише з переліку SortOrder, а не з тексту, введеного користувачем.

Лістинг 3.2 – Запит кількості працівників у підрозділах (клас JdbcDepartmentDao)

```
private static final String SELECT_WITH_COUNT = """
        SELECT d.id, d.name, d.location, d.phone, COUNT(e.id) AS employee_count
        FROM departments d
        LEFT JOIN employees e ON e.department_id = d.id
        GROUP BY d.id, d.name, d.location, d.phone
        ORDER BY employee_count %s, d.name
        """;


public List<DepartmentStats> findAllWithEmployeeCount(Connection c, SortOrder
order)
        throws SQLException {
    try (PreparedStatement ps =
c.prepareStatement(SELECT_WITH_COUNT.formatted(order.sql()));
         ResultSet rs = ps.executeQuery()) {
        List<DepartmentStats> result = new ArrayList<>();
        while (rs.next()) {
            result.add(new DepartmentStats(map(rs), rs.getInt("employee_count")));
        }
        return result;
    }
}
```

Фонд заробітної плати підрозділів обчислюється аналогічним запитом із агрегатною функцією COALESCE(SUM(e.salary), 0), а переведення працівників виконується одним оператором UPDATE employees SET department_id = ? WHERE department_id = ?.

3.3 Бізнес-логіка та перевірка даних

Клас Validator перевіряє об’єкти перед записом у базу і збирає всі помилки одразу, щоб користувач міг виправити їх за один раз. Перевіряються:

– обов’язковість полів і максимальна довжина рядків відповідно до розмірів полів у базі даних;

– прізвище, ім’я та по батькові – лише літери, апостроф, дефіс і пробіл;

– заробітна плата – число від 0 до 10 000 000 з не більше ніж двома знаками після коми (допускаються кома як роздільник і пробіли між розрядами);

– дата прийняття – не раніше 1950 року і не пізніше поточної дати;

– формат електронної пошти та телефону.

Сервіси додатково перевіряють правила, для яких потрібна база даних: унікальність назви підрозділу без урахування регістру, унікальність email,

існування вибраного підрозділу, заборону видалення підрозділу з працівниками.

Переведення всіх працівників (вимога на 85 балів) реалізовано в методі transferAllEmployees (лістинг 3.3). Перевірка існування обох підрозділів і оновлення виконуються в одній транзакції.

Лістинг 3.3 – Переведення працівників (клас DepartmentService)

```
public int transferAllEmployees(int fromId, int toId) {
    if (fromId == toId) {
        throw new ValidationException("Підрозділ-джерело і підрозділ-призначення
мають бути різними.");
    }
    return db.inTransaction(c -> {
        if (departmentDao.findById(c, fromId).isEmpty()) {
            throw new ValidationException("Підрозділ, з якого переводяться
працівники, не знайдено.");
        }
        if (departmentDao.findById(c, toId).isEmpty()) {
            throw new ValidationException("Підрозділ, до якого переводяться
працівники, не знайдено.");
        }
        int moved = employeeDao.moveAllToDepartment(c, fromId, toId);
        if (moved == 0) {
            throw new ValidationException("У вибраному підрозділі немає
працівників для переведення.");
        }
        return moved;
    });
}
```

Частка кожного підрозділу у фонді заробітної плати обчислюється у класі BigDecimal з округленням до двох знаків (лістинг 3.4). Якщо загальний фонд дорівнює нулю, частки дорівнюють нулю, що виключає ділення на нуль.

Лістинг 3.4 – Розрахунок загального фонду та часток (клас DepartmentService)

```
static SalaryFundReport buildReport(List<DepartmentFund> funds) {
    BigDecimal total = funds.stream()
            .map(DepartmentFund::fund)
            .reduce(BigDecimal.ZERO, BigDecimal::add)
            .setScale(2, RoundingMode.HALF_UP);
    List<SalaryFundReport.Entry> entries = new ArrayList<>();
    for (DepartmentFund f : funds) {
        BigDecimal share = total.signum() == 0
                ? BigDecimal.ZERO.setScale(2)
                : f.fund().multiply(HUNDRED).divide(total, 2,
RoundingMode.HALF_UP);
        entries.add(new SalaryFundReport.Entry(f.departmentId(),
f.departmentName(),
                f.employeeCount(), f.fund().setScale(2, RoundingMode.HALF_UP),
share));
    }
    return new SalaryFundReport(entries, total);
}
```

3.4 Експорт та імпорт JSON

Для вимоги на 100 балів реалізовано клас JsonExchangeService. Експорт зчитує всі підрозділи та працівників і записує їх у файл за допомогою бібліотеки Jackson. Працівник посилається на підрозділ за назвою, а не за числовим ідентифікатором, тому файл можна імпортувати в іншу базу даних, де ідентифікатори відрізняються. Фрагмент файлу наведено в лістингу 3.5.

Лістинг 3.5 – Фрагмент файлу експорту

```
{
  "version" : 1,
  "exportedAt" : "2026-11-11T14:05:31",
  "departments" : [ {
    "name" : "Юридичний відділ",
    "location" : "Корпус А, к. 201",
    "phone" : "+380542123405"
  }, ... ],
  "employees" : [ {
    "lastName" : "Павленко",
    "firstName" : "Ігор",
    "middleName" : "Степанович",
    "position" : "Юрисконсульт",
    "salary" : 40000.00,
    "hireDate" : "2016-12-01",
    "email" : "i.pavlenko@company.ua",
    "department" : "Юридичний відділ"
  }, ... ]
}
```

Імпорт виконується в одній транзакції. Підрозділи зіставляються з наявними за назвою, працівники – за email: наявні записи оновлюються, нові

додаються, тож повторний імпорт того самого файлу не створює дублікатів. Кожен запис перевіряється класом Validator. Якщо хоча б один запис некоректний або посилається на неіснуючий підрозділ, програма показує перелік усіх помилок із номерами записів, а транзакція відкочується, і база даних залишається без змін. Пошкоджений або порожній файл також призводить до зрозумілого повідомлення.

3.5 Графічний інтерфейс користувача

Головне вікно містить меню «Файл» (експорт у JSON – Ctrl+E, імпорт з JSON – Ctrl+I, оновлення – F5, зміна підключення, вихід), меню «Довідка» і три вкладки.

Вкладка «Підрозділи» (рисунок 3.2) у верхній частині показує таблицю підрозділів із кількістю працівників і список для вибору порядку сортування «За спаданням» або «За зростанням». При виборі підрозділу в нижній частині вікна з’являється список його працівників. Кнопки дозволяють додати, редагувати чи видалити підрозділ, перевести всіх його працівників до іншого підрозділу, а також керувати працівниками вибраного підрозділу.

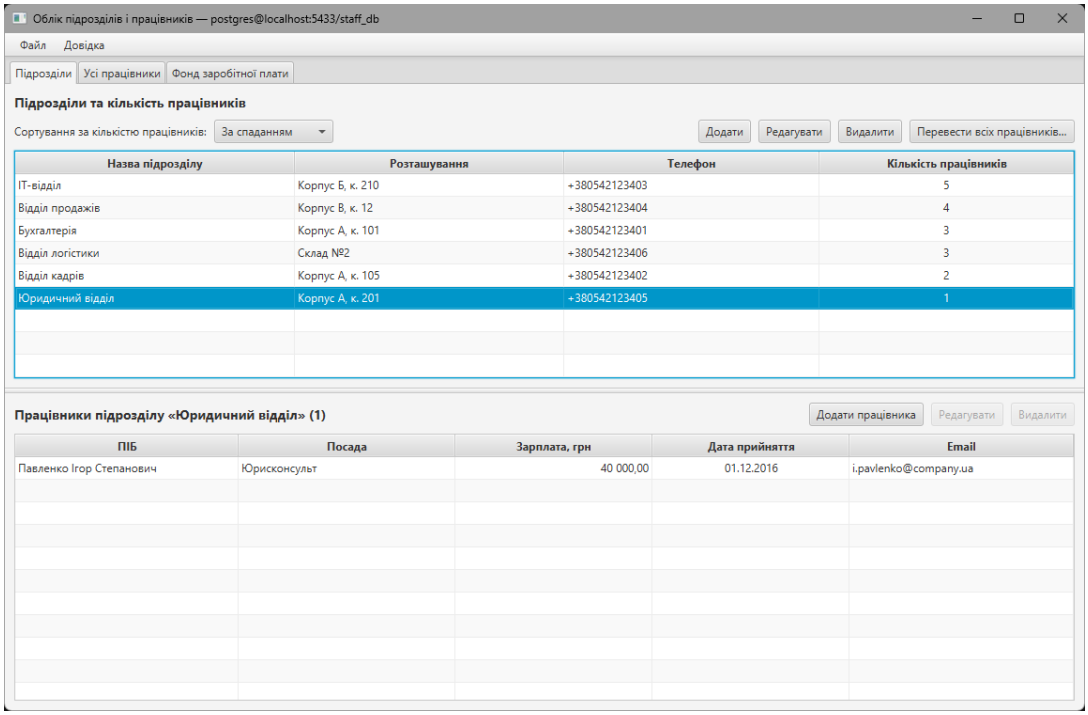


Рисунок 3.2 – Вкладка «Підрозділи»

Вкладка «Усі працівники» (рисунок 3.3) містить таблицю всіх працівників з пошуком за ПІБ, посадою, підрозділом або email і сортуванням за будь-яким стовпцем.

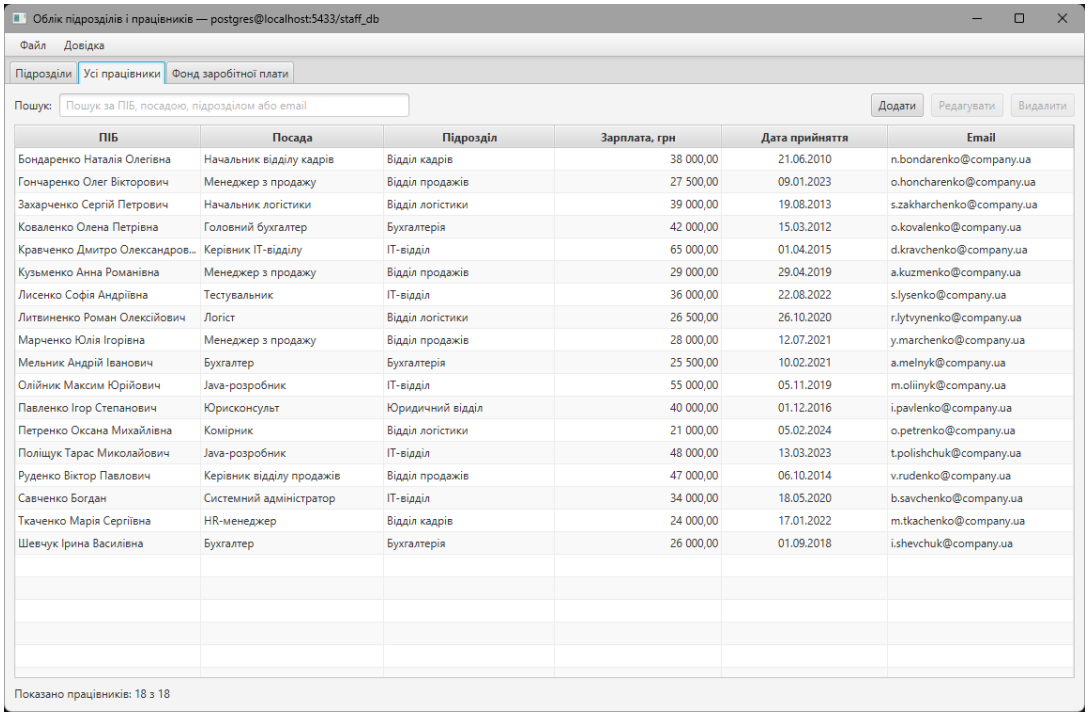


Рисунок 3.3 – Вкладка «Усі працівники»

Додавання та редагування працівника виконується в діалоговому вікні (рисунок 3.4). Дата вибирається з календаря або вводиться у форматі дд.мм.рррр, підрозділ – зі списку. Якщо дані некоректні, вікно не закривається, а користувач отримує перелік помилок.

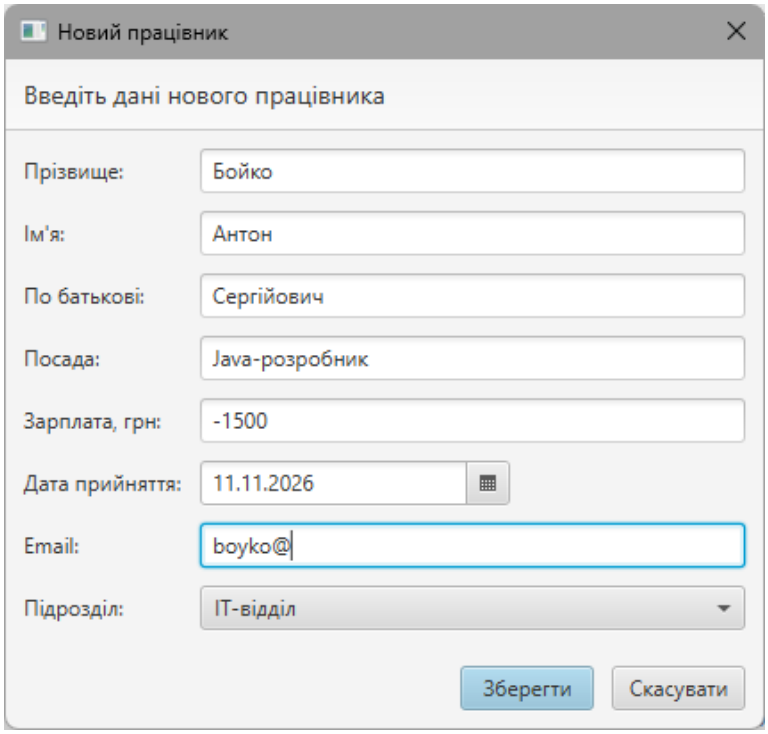


Рисунок 3.4 – Діалог додавання працівника

Вкладка «Фонд заробітної плати» (рисунок 3.5) показує для кожного підрозділу кількість працівників, суму заробітної плати і частку в загальному фонді, а також загальний фонд і секторну діаграму розподілу.

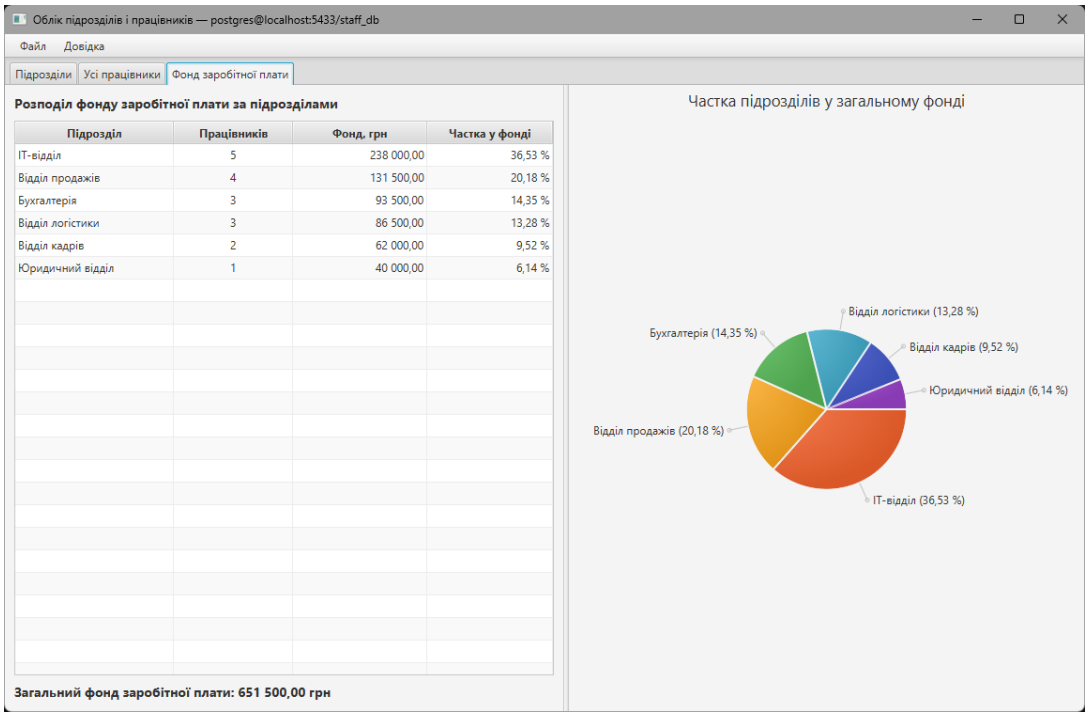


Рисунок 3.5 – Вкладка «Фонд заробітної плати»

3.6 Обробка винятків

Помилки бази даних (SQLException) не показуються користувачу у «сирому» вигляді. Клас SqlErrors перетворює їх на виняток DataAccessException із повідомленням українською мовою за кодом SQLSTATE (таблиця 3.1). Помилки перевірки даних передаються винятком ValidationException зі списком усіх знайдених помилок. Інтерфейс перехоплює ці винятки та показує діалог помилки, а непередбачені винятки обробляє глобальний обробник, тож програма не завершується аварійно.

Таблиця 3.1 – Переклад кодів помилок PostgreSQL

SQLSTATE	Ситуація	Повідомлення користувачу
23505	Порушення унікальності	Запис з такими даними вже існує
23503	Порушення зовнішнього ключа	Запис пов’язаний з іншими даними
23502	Порожнє обов’язкове поле	Не заповнено обов’язкове поле
23514	Порушення CHECK	Значення не відповідає обмеженням бази даних
28P01, 28000	Помилка автентифікації	Невірний логін або пароль
3D000	Немає бази даних	Базу даних з такою назвою не знайдено
42P01	Немає таблиці	Виконайте скрипт sql/schema.sql
08xxx	Немає з’єднання	Не вдалося з’єднатися із сервером PostgreSQL

4 ТЕСТУВАННЯ

4.1 Функціональне тестування інтерфейсу

Ручне тестування виконувалося на базі даних із тестовими даними зі скрипту sql/data.sql: 6 підрозділів, 18 працівників, загальний фонд заробітної плати 651 500,00 грн. Результати наведено в таблиці 4.1.

Таблиця 4.1 – Результати функціонального тестування

№	Перевірка	Очікуваний результат	Результат
1	Вхід з правильними даними	Відкривається головне вікно	Пройдено
2	Вхід до неіснуючої бази даних або недоступного сервера	Повідомлення про помилку, вікно входу залишається	Пройдено
3	Перегляд підрозділів, сортування «За спаданням»	Кількості 5, 4, 3, 3, 2, 1	Пройдено
4	Сортування «За зростанням»	Кількості 1, 2, 3, 3, 4, 5	Пройдено
5	Вибір підрозділу «Юридичний відділ»	Показано 1 працівника (рис. 3.2)	Пройдено
6	Додавання працівника до IT-відділу	Кількість змінилася з 5 на 6	Пройдено
7	Введення від’ємної зарплати та неправильного email	Обидві помилки показано разом, запис не збережено (рис. 4.1)	Пройдено
8	Переведення всіх працівників IT-відділу до бухгалтерії	Підтвердження, IT-відділ має 0 працівників, бухгалтерія – 9 (рис. 4.2)	Пройдено
9	Видалення підрозділу з працівниками	Повідомлення про заборону видалення	Пройдено
10	Фонд заробітної плати	Загальний фонд 651 500,00 грн, частка IT-відділу 36,53 %	Пройдено
11	Експорт у JSON і імпорт у порожню базу	Відновлено 6 підрозділів і 18 працівників	Пройдено
12	Імпорт файлу з некоректним записом	Перелік помилок, база не змінилася	Пройдено

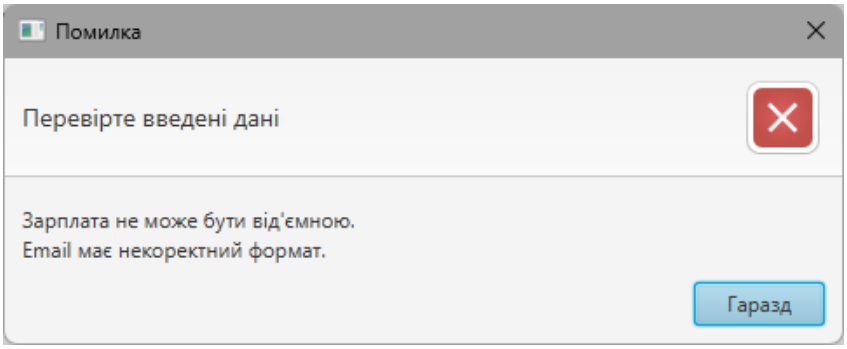


Рисунок 4.1 – Повідомлення про некоректні дані

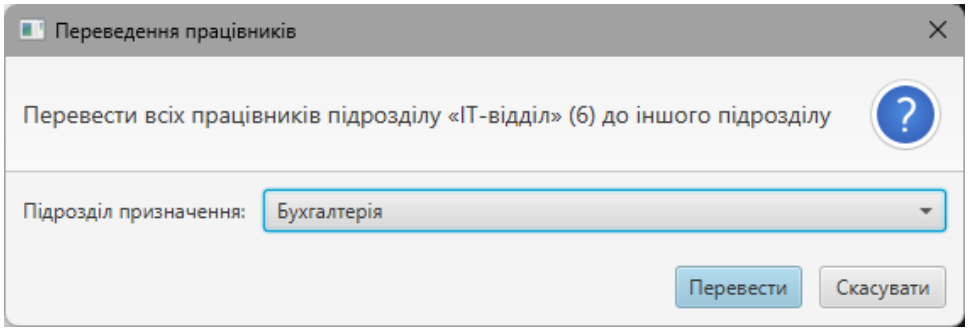


Рисунок 4.2 – Діалог переведення працівників

Частки підрозділів у фонді заробітної плати для тестових даних наведено в таблиці 4.2. Їхня сума дорівнює 100 %, що підтверджує правильність розрахунку.

Таблиця 4.2 – Фонд заробітної плати для тестових даних

Підрозділ	Працівників	Фонд, грн	Частка, %
IT-відділ	5	238 000,00	36,53
Відділ продажів	4	131 500,00	20,18
Бухгалтерія	3	93 500,00	14,35
Відділ логістики	3	86 500,00	13,28
Відділ кадрів	2	62 000,00	9,52
Юридичний відділ	1	40 000,00	6,14
Разом	18	651 500,00	100,00

4.2 Автоматизовані тести

Для перевірки логіки написано 29 автоматизованих тестів на JUnit 6. Тести, що працюють із базою даних, використовують бібліотеку Embedded PostgreSQL: вона запускає справжній сервер PostgreSQL, а перед кожним тестом база заново створюється скриптами sql/schema.sql і sql/data.sql. Отже, перевіряються реальні SQL-запити, обмеження та транзакції. Перелік тестів наведено в таблиці 4.3.

Таблиця 4.3 – Автоматизовані тести

Клас тестів	Що перевіряється	К-сть
`ValidatorTest`	Коректні дані та обрізання пробілів; апостроф і дефіс в іменах; збирання всіх помилок одразу; зарплата з більш ніж двома знаками після коми; розбір зарплати з комою та пробілами; перевірка підрозділу	6

Клас тестів	Що перевіряється	К-сть
`DepartmentServiceTest`	Кількість працівників у підрозділах; підрозділи без працівників; сортування; переведення працівників; заборона переведення в той самий або з порожнього підрозділу; фонд заробітної плати і частки; нульовий фонд; заборона видалення підрозділу з працівниками; унікальність назви; редагування	10
`EmployeeServiceTest`	Додавання; повторний email; некоректні дані; неіснуючий підрозділ; редагування і видалення	5
`JsonExchangeServiceTest`	Експорт та імпорт у порожню базу; повторний імпорт без дублікатів; відкат усього імпорту через один некоректний запис; невідомий підрозділ і пошкоджений JSON	4
`DatabaseTest`	Підключення з правильними даними; неіснуюча база; недоступний сервер; відкат транзакції при помилці	4
`Усього`		29

Тести запускаються командою mvn test. Усі 29 тестів пройдено успішно.

ВИСНОВКИ

У курсовій роботі розроблено настільний застосунок «Облік підрозділів і працівників» мовою Java з графічним інтерфейсом JavaFX і базою даних PostgreSQL.

Спроєктовано базу даних із двох пов’язаних таблиць з обмеженнями первинного та зовнішнього ключів, унікальності та CHECK, які забезпечують цілісність даних незалежно від програми. Застосунок має багаторівневу архітектуру: модель, доступ до даних (DAO), бізнес-логіку (сервіси) та інтерфейс користувача. Користувач не працює з SQL безпосередньо, а всі запити виконуються через PreparedStatement.

Виконано всі вимоги варіанта 4:

– виведення підрозділів із кількістю працівників, перегляд працівників вибраного підрозділу та сортування за чисельністю;

– переведення всіх працівників підрозділу до іншого в одній транзакції;

– розрахунок загального фонду заробітної плати і частки кожного підрозділу з таблицею та діаграмою;

– експорт та імпорт даних у форматі JSON з перевіркою записів і повним відкатом у разі помилок.

Програма перевіряє введені дані, показує зрозумілі повідомлення про помилки українською мовою і коректно обробляє збої підключення до бази даних. Працездатність підтверджено ручним тестуванням і 29 автоматизованими тестами на справжньому сервері PostgreSQL.

Подальший розвиток можливий у напрямі розмежування прав користувачів, ведення історії переведень працівників і формування звітів у форматах PDF або Excel.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1) Шилдт Г. Java. Повне керівництво. 12-те вид. – Київ : Діалектика, 2023. – 1408 с.

2) Блох Дж. Java. Ефективне програмування. 3-тє вид. – Київ : Діалектика, 2019. – 464 с.

3) The Java Tutorials. JDBC Database Access [Електронний ресурс]. – Oracle. – Режим доступу: https://docs.oracle.com/javase/tutorial/jdbc/

4) JavaFX Documentation [Електронний ресурс]. – OpenJFX. – Режим доступу: https://openjfx.io/

5) PostgreSQL Documentation [Електронний ресурс]. – The PostgreSQL Global Development Group. – Режим доступу: https://www.postgresql.org/docs/

6) PostgreSQL JDBC Driver Documentation [Електронний ресурс]. – Режим доступу: https://jdbc.postgresql.org/documentation/

7) Jackson Project [Електронний ресурс]. – FasterXML. – Режим доступу: https://github.com/FasterXML/jackson

8) JUnit User Guide [Електронний ресурс]. – Режим доступу: https://docs.junit.org/

9) Apache Maven Project [Електронний ресурс]. – Режим доступу: https://maven.apache.org/

ДОДАТОК А
ІНСТРУКЦІЯ ІЗ ЗАПУСКУ

Вихідний код проєкту: https://github.com/CCh1e/staff-coursework. Отримати його можна командою git clone https://github.com/CCh1e/staff-coursework.git або кнопкою «Code → Download ZIP» на сторінці репозиторію.

Для роботи потрібні JDK 25 або новіший, Apache Maven 3.9 і сервер PostgreSQL.

1. Створення бази даних. У psql або pgAdmin виконати:

```
CREATE DATABASE staff_db;
\c staff_db
\i sql/schema.sql
\i sql/data.sql
```

2. Запуск програми з каталогу проєкту:

```
mvn javafx:run
```

або зібрати один виконуваний JAR-файл і запустити його:

```
mvn package
java -jar target/staff-coursework-1.0.0-all.jar
```

3. Вхід. У вікні входу вказати сервер localhost, порт 5432, базу даних staff_db, ім’я користувача та пароль PostgreSQL.

4. Демонстрація без встановленого PostgreSQL. Клас ua.sumdu.staff.DemoDatabase з тестового коду запускає вбудований сервер PostgreSQL на порту 5433 з базою staff_db і тестовими даними. Після його запуску в програмі слід вказати порт 5433, користувача postgres і порожній пароль.

5. Автоматизовані тести: mvn test.